



Lista 2 — JavaScript e Manipulação do DOM

Aulas 11 a 20

Desenvolvimento Web

Exercícios sobre fundamentos do JavaScript (ES6+), manipulação do DOM, eventos, formulários e validação. Cenários de interatividade em aplicações web.

Exercício 1

Cenário: Cliente quer calcular o IMC dos usuários no site de uma clínica de nutrição.

1. Crie uma calculadora de IMC em JS: formulário com inputs de peso (kg) e altura (m), botão calcular, exibição do resultado e classificação (abaixo do peso, normal, sobrepeso, obeso).
2. Use arrow functions para o cálculo e exibição. Trate entradas inválidas (NaN, valores negativos, zero).
3. Adicione validação visual: borda verde em campos válidos, vermelha em inválidos, mensagem de erro abaixo do campo.
4. Explique event loop do JS: dado `setTimeout(()=>console.log(3), 0)` e `Promise.resolve().then(()=>console.log(2))` e `console.log(1)`, qual a ordem? Por quê?

Exercício 2

Cenário: Dashboard de vendas com dados dinâmicos — atualizar tabela com novos registros sem recarregar a página.

1. Crie uma tabela HTML com cabeçalho (Produto, Quantidade, Preço, Total) e use JS para renderizar dados de um array de objetos.
2. Adicione um formulário para inserir novos produtos e atualize a tabela dinamicamente (sem innerHTML, use `createElement` + `appendChild`).
3. Implemente event delegation: um único event listener no tbody captura cliques nos botões "Remover" de cada linha.
4. Calcule o total geral usando `reduce()` e exiba no final da tabela. Atualize sempre que um item for adicionado ou removido.

Exercício 3

Cenário: Formulário de cadastro com validação em tempo real e integração com API de CEP.

1. Campos: nome, email, CPF, CEP, endereço, número, bairro, cidade, UF. Use `fetch()` para preencher endereço automaticamente ao digitar o CEP (ViaCEP API: <https://viacep.com.br/ws/{CEP}/json/>).
2. Valide: (a) email com regex, (b) CPF com algoritmo de dígitos verificadores, (c) CEP com formato 00000-000.
3. Adicione feedback visual: spinner de loading enquanto a API responde, campos desabilitados durante a requisição.
4. Pesquise `async/await` vs Promises: reescreva a chamada `fetch` com ambas as sintaxes. Qual é mais legível em fluxos com múltiplas requisições dependentes?

Exercício 4

Cenário: Carrinho de compras com armazenamento local (`localStorage`).

1. Modele o estado do carrinho como array de objetos `{id, nome, preco, qtd}`. Renderize os itens em uma lista.
2. Implemente: adicionar ao carrinho, remover, alterar quantidade, calcular subtotal e total.
3. Persista o carrinho no `localStorage`: salve a cada alteração e carregue ao iniciar a página.
4. Explique closures em JS: crie uma função `criarContador()` que retorna `{incrementar, decrementar, getValor}` e demonstre o encapsulamento do contador.

Requisitos

Navegador (Chrome DevTools), Node.js, VSCode. ViaCEP API (gratuita, sem chave).